

- Description

This module manages a smart water resource allocation system, ensuring that:

1. **Water consumption is monitored across different sectors (agriculture, industry, residential).**
2. **Water distribution is prioritized based on sector importance and availability.**
3. **If total demand exceeds supply, the system enforces rationing policies.**
4. **A log of all water allocations and shortages** is maintained for future analysis.

This system ensures efficient water distribution, prevents shortages, and optimizes resource allocation dynamically.

- Specification

module SmartWaterAllocation

var totalWaterSupply: real

var waterRequests: sequence of (string, real)

var allocationLog: sequence of string

process allocateWater(waterRequests: sequence of (string, real))

functional scenario form:

 totalWaterRequests <= totalWaterSupply && totalWaterSupply = totalWaterSupply - sum(waterRequests) and allocationLog = allocationLog $\oplus$ "Water allocated successfully", Water allocated successfully.

 ||

 totalWaterRequests > totalWaterSupply && totalWaterSupply = 0 and allocationLog = allocationLog $\oplus$ "Water rationing applied", Water supply insufficient, rationing applied.

end process

end module

- Java Code

```
package code;
import java.util.*;
class WaterRequest {
    private String sector;
    private double requestedAmount;
    public WaterRequest(String sector, double requestedAmount) {
        this.sector = sector;
        this.requestedAmount = requestedAmount;
    }
    public String getSector() {
        return sector;
    }
    public double getRequestedAmount() {
        return requestedAmount;
    }
}
```

```

}
class WaterResourceManager {
    private double totalWaterSupply;
    private Map<String, Double> sectorPriority;
    private List<String> allocationLog;
    public WaterResourceManager(double totalWaterSupply) {
        this.totalWaterSupply = totalWaterSupply;
        this.sectorPriority = new HashMap<>();
        this.allocationLog = new ArrayList<>();
        sectorPriority.put("Agriculture", 1.5);
        sectorPriority.put("Residential", 1.0);
        sectorPriority.put("Industry", 0.8);
    }
    public void allocateWater(List<WaterRequest> requests) {
        double totalDemand = requests.stream().mapToDouble(WaterRequest
::getRequestedAmount).sum();
        if (totalDemand <= totalWaterSupply) {
            for (WaterRequest req : requests) {
                allocateSectorWater(req.getSector(), req.getRequestedAm
ount());
            }
        } else {
            System.out.println("Water supply insufficient! Implementing
rationing...");
            enforceRationing(requests, totalDemand);
        }
    }
    private void allocateSectorWater(String sector, double amount) {
        totalWaterSupply -= amount;
        allocationLog.add(String.format("Allocated: %.2f cubic meters t
o %s", amount, sector));
        System.out.printf("Allocated %.2f cubic meters to %s\n", amount
, sector);
    }
    private void enforceRationing(List<WaterRequest> requests, double t
otalDemand) {
        double scalingFactor = totalWaterSupply / totalDemand;

        requests.sort((a, b) -> Double.compare(sectorPriority.get(b.get
Sector()), sectorPriority.get(a.getSector())));
        for (WaterRequest req : requests) {
            double allocatedAmount = req.getRequestedAmount() * scaling
Factor * sectorPriority.get(req.getSector());
            allocateSectorWater(req.getSector(), allocatedAmount);

```

```

    }
}
public void printAllocationLog() {
    System.out.println("\nWater Allocation Log:");
    for (String log : allocationLog) {
        System.out.println(log);
    }
}
}

public class c14_SmartWaterSystem {
    public static void main(String[] args) {

//      //testcase-VT:
//      WaterResourceManager manager1 = new WaterResourceManager(500);

//      List<WaterRequest> requests1 = Arrays.asList(
//          new WaterRequest("Agriculture", 489),
//          new WaterRequest("Industry", 479),
//          new WaterRequest("Residential", 521)
//      );
//      manager1.allocateWater(requests1);
//      manager1.printAllocationLog();
//
//      WaterResourceManager manager2 = new WaterResourceManager(500);

//      List<WaterRequest> requests2 = Arrays.asList(
//          new WaterRequest("Agriculture", -234),
//          new WaterRequest("Industry", -24),
//          new WaterRequest("Residential", 0)
//      );
//      manager2.allocateWater(requests2);
//      manager2.printAllocationLog();
//
//      WaterResourceManager manager3 = new WaterResourceManager(500);

//      List<WaterRequest> requests3 = Arrays.asList(
//          new WaterRequest("Agriculture", 327),
//          new WaterRequest("Industry", 21),
//          new WaterRequest("Residential", 94)
//      );
//      manager3.allocateWater(requests3);
//      manager3.printAllocationLog();

//testcase-FT:

```

```

//      WaterResourceManager manager1 = new WaterResourceManager(500);

//      List<WaterRequest> requests1 = Arrays.asList(
//          new WaterRequest("Agriculture", 4.185580251239443E298),
//          new WaterRequest("Industry", 0),
//          new WaterRequest("Residential", 0)
//      );
//      manager1.allocateWater(requests1);
//      manager1.printAllocationLog();
//
//      WaterResourceManager manager2 = new WaterResourceManager(500);

//      List<WaterRequest> requests2 = Arrays.asList(
//          new WaterRequest("Agriculture", -1.7976931348623157E308),
//          new WaterRequest("Industry", 0),
//          new WaterRequest("Residential", 0)
//      );
//      manager2.allocateWater(requests2);
//      manager2.printAllocationLog();
//
//      WaterResourceManager manager3 = new WaterResourceManager(500);

//      List<WaterRequest> requests3 = Arrays.asList(
//          new WaterRequest("Agriculture", 0),
//          new WaterRequest("Industry", 4.9E-324),
//          new WaterRequest("Residential", 1.559333923860884E298)
//      );
//      manager3.allocateWater(requests3);
//      manager3.printAllocationLog();

//testcase-Z3:
//      WaterResourceManager manager1 = new WaterResourceManager(500);

//      List<WaterRequest> requests1 = Arrays.asList(
//          new WaterRequest("Agriculture", 0),
//          new WaterRequest("Industry", 0),
//          new WaterRequest("Residential", 0)
//      );
//      manager1.allocateWater(requests1);
//      manager1.printAllocationLog();
//
//      WaterResourceManager manager2 = new WaterResourceManager(500);

//      List<WaterRequest> requests2 = Arrays.asList(

```

```

//      new WaterRequest("Agriculture", -1),
//      new WaterRequest("Industry", -1),
//      new WaterRequest("Residential", -1)
//  );
//  manager2.allocateWater(requests2);
//  manager2.printAllocationLog();
//
//  WaterResourceManager manager3 = new WaterResourceManager(500);

//  List<WaterRequest> requests3 = Arrays.asList(
//      new WaterRequest("Agriculture", 501),
//      new WaterRequest("Industry", 501),
//      new WaterRequest("Residential", 501)
//  );
//  manager3.allocateWater(requests3);
//  manager3.printAllocationLog();

//testcase-UVT:
WaterResourceManager manager1 = new WaterResourceManager(500);
List<WaterRequest> requests1 = Arrays.asList(
    new WaterRequest("Agriculture", 489),
    new WaterRequest("Industry", 479),
    new WaterRequest("Residential", 521)
);
manager1.allocateWater(requests1);
manager1.printAllocationLog();

WaterResourceManager manager2 = new WaterResourceManager(500);
List<WaterRequest> requests2 = Arrays.asList(
    new WaterRequest("Agriculture", -234),
    new WaterRequest("Industry", -24),
    new WaterRequest("Residential", 0)
);
manager2.allocateWater(requests2);
manager2.printAllocationLog();

WaterResourceManager manager3 = new WaterResourceManager(500);
List<WaterRequest> requests3 = Arrays.asList(
    new WaterRequest("Agriculture", 327),
    new WaterRequest("Industry", 21),
    new WaterRequest("Residential", 94)
);
manager3.allocateWater(requests3);
manager3.printAllocationLog();

```

```

    }
}

```

- Coverage report

1. Test case number: 3

2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	826	834
✓ src	1.0 %	8	826	834
✓ code	1.0 %	8	826	834
✓ c14_SmartWaterSystem.java	100.0 %	8	0	8
> c14_SmartWaterSystem		0	0	0
> WaterRequest		0	0	0
> WaterResourceManager	100.0 %	8	0	8

3. Vibration testing instruction coverage: 99.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	2.0 %	317	15,840	16,157
✓ src	2.0 %	317	15,840	16,157
✓ code	2.0 %	317	15,840	16,157
✓ c14_SmartWaterSystem.java	99.1 %	317	3	320
> c14_SmartWaterSystem	97.5 %	115	3	118
> WaterRequest	100.0 %	15	0	15
> WaterResourceManager	100.0 %	187	0	187

4. Fuzz testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	826	834
✓ src	1.0 %	8	826	834
✓ code	1.0 %	8	826	834
✓ c14_SmartWaterSystem.java	100.0 %	8	0	8
> c14_SmartWaterSystem		0	0	0
> WaterRequest		0	0	0
> WaterResourceManager	100.0 %	8	0	8

5. Fuzz testing instruction coverage: 99.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	2.0 %	317	15,840	16,157
✓ src	2.0 %	317	15,840	16,157
✓ code	2.0 %	317	15,840	16,157
✓ c14_SmartWaterSystem.java	99.1 %	317	3	320
> c14_SmartWaterSystem	97.5 %	115	3	118
> WaterRequest	100.0 %	15	0	15
> WaterResourceManager	100.0 %	187	0	187

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	832	840
✓ src	1.0 %	8	832	840
✓ code	1.0 %	8	832	840
✓ c14_SmartWaterSystem.java	100.0 %	8	0	8
> WaterResourceManager	100.0 %	8	0	8
> c14_SmartWaterSystem		0	0	0
> WaterRequest		0	0	0

7. Z3 instruction coverage: 99.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.4 %	317	21,776	22,093
✓ src	1.4 %	317	21,776	22,093
✓ code	1.4 %	317	21,776	22,093
✓ c14_SmartWaterSystem.java	99.1 %	317	3	320
> c14_SmartWaterSystem	97.5 %	115	3	118
> WaterRequest	100.0 %	15	0	15
> WaterResourceManager	100.0 %	187	0	187

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	832	840
▼ src	1.0 %	8	832	840
▼ code	1.0 %	8	832	840
▼ c14_SmartWaterSystem.java	100.0 %	8	0	8
> WaterResourceManager	100.0 %	8	0	8
> c14_SmartWaterSystem		0	0	0
> WaterRequest		0	0	0

9. Unimproved V-method instruction coverage: 99.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.4 %	317	21,776	22,093
▼ src	1.4 %	317	21,776	22,093
▼ code	1.4 %	317	21,776	22,093
▼ c14_SmartWaterSystem.java	99.1 %	317	3	320
> c14_SmartWaterSystem	97.5 %	115	3	118
> WaterRequest	100.0 %	15	0	15
> WaterResourceManager	100.0 %	187	0	187